School of Computing and Mathematical Sciences

CO7201 Individual Project

Final Report

Software Bug Tracking & Reporting Tool

Parth Bharat Maru

pbm5@student.le.ac.uk

249040206

Project Supervisor: Dr. Martina Palusa
Principal Marker: Dr. Anand Sengodan

Word Count: 7759
15/08/2025

DECLARATION

All the sentences or passages quoted in this report, or computer code of any form whatsoever used and/ or submitted at any stages, which are taken from other people's work have been specifically acknowledged by clear citation of the source, specifying author, work, date and page(s). Any part of my own written work, or software coding, which is substantially based upon other people's work, is duly accompanied by clear citation of the source, specifying author, work, date and page(s). I understand that failure to do this amounts to plagiarism and will be considered grounds for failure in this module and the degree examination as a whole.

Name: Parth Bharat Maru
Date: 15/08/2025

Contents

1. Introduction 5
  1.1 Motivation 5
  1.2 Aims and objectives 5
  1.3 Challenges 6
  1.4 Risks 7
  1.5 System Overview 8
2. Background Research 9
  2.1 Technical Research and Technology Justification 9
    2.1.1 Frontend: React, TypeScript, Tailwind CSS, shadcn/ui, and Radix 9
    2.1.2 Backend: Spring Boot (Java 17) 10
    2.1.3 Database: PostgreSQL with Flyway Migrations 10
    2.1.4 Security: JWT Authentication and Role-Based Access Control 10
    2.1.5 Advanced Feature Technologies 10
      Duplicate Bug Detection 10
      Automatic Team Assignment 10
      Automatic User Assignment 11
    Technological Justification 11
  2.2 Literature Review 11
    2.2.1 Jira 11
    2.2.2 Bugzilla 12
    2.2.3 Redmine 12
    2.2.4 GitHub Issues 13
  2.3 Identified Gaps and Justification for the Proposed System 13
3. Requirements 13

3.1. Essential Requirements (Must have) 13
3.2. Required Requirements (Should have) 14
3.3. Optional Requirements (Could have) 14
4. Technical Specifications 15
4.1. Frontend 15
4.2 Backend 15
4.3 Database and Data Management 16
4.4 Advanced Feature Implementations 16
4.4.1. Duplicate Bug Detection 16
4.4.2. Automatic Team Allocation 16
4.4.3. Automatic User Allocation 16
4.5. Security 16
4.6. Development Environment and Tooling 17
4.7. Deployment 17
5. Design 17
5.1. Client Server Architecture Design 17
5.2. Database Design 19
5.3. Duplicate Bug Detection Design 19
5.4. Auto Team Assignment To Bug 22
5.5. Auto user assignment based on skills 23
6. Implementation 25
6.1. Duplicate Bug Detection 25
Where the check is triggered 25
Project-level configuration 26
Preprocessing 27
TF-IDF vector building 27
Candidate selection 27
Similarity scoring 28
Combining scores and applying threshold 28
Caching and returning results 28
Frontend display 28
Why this implementation was chosen 29
Current limitations and next steps 29
7. Critical Reflection and Evaluation 29
7.1. Challenges faced during development 29
8. Conclusion 30
9. Recommendations for future work 31
10. References 32

1. Introduction

1.1 Motivation

In modern software development, managing bugs effectively is essential for keeping projects on track. When bugs are not reported or tracked properly, they can cause delays, reduce product quality, and increase costs. Many teams still rely on scattered methods such as spreadsheets, email chains, or informal communication, which makes it difficult to keep track of progress and responsibilities. This project aims to address that challenge by creating a central platform where bugs can be reported, assigned, and monitored in an organised way.

A dedicated bug tracking tool can also improve communication between developers, testers, and project managers. By having all the information in one place, teams can work more efficiently and avoid misunderstandings. This is especially important in collaborative environments where multiple people are working on different parts of a system. With clear records of each bug's status, history, and assigned team members, everyone can stay informed without constant back-and-forth messages.

Another motivation for this project comes from the need for features that go beyond basic bug reporting. Modern tools need to help teams prioritise issues, detect duplicates, and analyse patterns in reported bugs. These insights can guide decision making, help allocate resources better, and even prevent similar bugs from happening again. By integrating features such as duplicate detection and analytics, this project aims to offer a more intelligent and proactive approach to bug management.

The motivation is also shaped by my own professional experience. Having worked for three years as a production support engineer, I was directly involved in the bug management lifecycle documenting root cause analyses, resolving tickets, and coordinating with different teams to move issues towards closure. During this time, I extensively used JIRA and gained first-hand knowledge of its strengths as well as the areas where improvements could be made. This background has strongly influenced the design of this project, as I have drawn on real-world workflows and challenges to ensure the tool is practical and relevant.

1.2 Aims and objectives

The main aim of this project is to design and build a secure, easy-to-use, and modular web-based bug tracking tool. The system should allow different users such as developers, testers, and administrators to report bugs, track their progress, and manage their resolution in a structured way. It is intended to support clear communication, improve productivity, and provide useful insights into the bug resolution process.
To achieve this aim, the project will focus on the following objectives:

1. **Requirement analysis and planning:** Identify the main user roles and their permissions, decide the minimum features needed for the first working version, and plan the work around set deadlines.

2. **Frontend development:** Create responsive pages using React, TypeScript, and Tailwind CSS for user registration, login, project and team management, and bug reporting. Include protected routes and token handling to ensure secure access.
3. **Backend development:** Build RESTful APIs using Spring Boot for authentication, project and team creation, bug management, and analytics. Secure all endpoints using JWT-based authentication and apply proper input validation and error handling.
4. **Database design:** Set up a PostgreSQL database with a well-structured schema for users, teams, projects, bugs, and related data. Use Flyway to manage database versioning and updates.
5. **Role-based access control:** Implement permissions so that each role can only access the features and data relevant to their responsibilities.
6. **Testing and validation:** Carry out unit tests and integration tests for backend services, and perform manual testing of the complete workflow from the frontend to the backend.
7. **Documentation:** Maintain clear technical documentation for the architecture, APIs, and module interactions, along with setup guides for future developers or users.
8. **Advanced features:** Add functionality that goes beyond basic bug tracking, including:
    - Duplicate bug detection.
    - Automatic team allocation based on bug labels.
    - Automatic user allocation based on bug tags and user skills such as Java, PostgreSQL, etc.
    - Smart user allocation based on the user workload.
    - Gamification features like user leaderboards and a points/rewards system for bugs resolved.
    - Bug analytics to provide insights into trends, priorities, and performance.

By completing these objectives, the project will deliver a practical and fully functional bug tracking system that meets academic requirements while also being relevant for real-world use.

1.3 Challenges

Developing this project has involved working with a wide range of tools, frameworks, and concepts, many of which required learning and adaptation during the process. Some of the main challenges faced are outlined below.

1. Implementing Duplicate Bug Detection

Although not based on complex NLP libraries, designing an effective duplicate detection feature in Java was challenging. It required understanding and comparing different similarity algorithms such as Cosine Similarity, Jaccard Index, and Levenshtein Distance, and adapting them to work with bug titles and descriptions. Fine-tuning these methods to balance performance and accuracy for different project contexts took considerable testing and adjustments.

2. Automatic Team and User Allocation
The automatic allocation features for teams and users were technically demanding. The team allocation logic had to consider bug labels, while user allocation had to match bug tags with stored user skill sets (e.g., Java, PostgreSQL). This required designing a matching process that could handle incomplete or inconsistent data, while still producing useful and fair recommendations.

3. Ensuring Secure Role-Based Access Control
The system uses JWT-based authentication and strict role-based access rules. While this is a common approach, integrating it cleanly with the frontend routes, backend controllers, and database models took careful planning. Edge cases such as preventing unauthorised users from accessing sensitive actions like assigning bugs or changing project settings needed thorough testing.

4. Balancing Manual and Automated Workflows
For certain features like bug assignment, it was important to give administrators the ability to override automated recommendations. Designing a workflow that supports both manual control and automated allocation without causing conflicts or inconsistencies required additional logic and clear user interface feedback.

5. Gamification and Analytics Features
The gamification system including leaderboards and reward points had to be integrated in a way that stayed consistent with bug lifecycle events. Similarly, the analytics module required collecting and processing bug-related data in a way that was efficient but still accurate. Both features introduced challenges in designing data queries and ensuring the results remained up to date with minimal performance impact.

6. Frontend–Backend Synchronisation
Because development was carried out module-by-module, ensuring that each new frontend feature worked smoothly with the backend APIs required frequent integration checks. Small mismatches in field names, request structures, or validation rules could lead to errors, which sometimes took time to track down and fix.

1.4 Risks

While planning and developing the Bug Tracking and Reporting Tool, several risks were identified that could affect the quality, scope, or delivery timeline. These were monitored throughout the development process to minimise their impact.

1. Learning curve for certain technologies
Although I had prior experience with Java and SQL, some technologies used in this project such as React with TypeScript, Tailwind CSS, and Flyway database migrations were new to me. This required additional learning time, and unexpected implementation issues occasionally slowed down progress.

2. Advanced feature complexity
Some planned features, such as duplicate bug detection and automatic user allocation, introduced additional complexity. The duplicate detection algorithm still needs refinement, and the user allocation logic requires more testing. These features risked consuming more time than expected, which could have delayed other planned tasks.

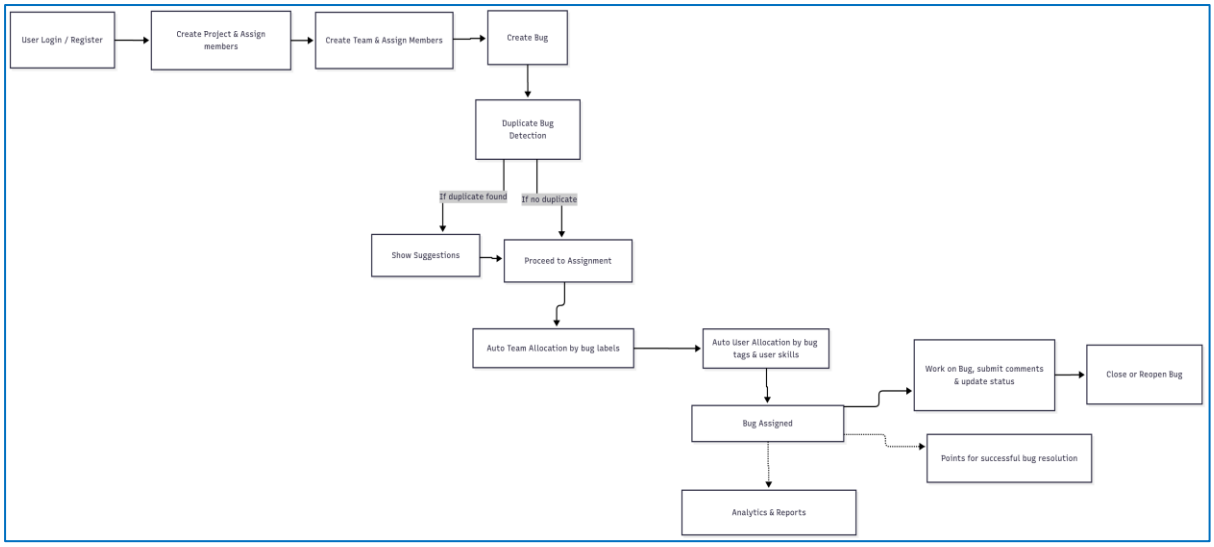
3. Integration challenges between frontend and backend
The project was developed in separate modules, which meant that keeping the frontend and backend in sync was an ongoing risk. Even small mismatches in API structures or field names sometimes caused integration failures, leading to rework.

4. Limited automated testing
While manual testing has been carried out regularly, automated testing has not yet been fully implemented. This creates a risk of undetected regressions when changes are made, especially as the system becomes more complex.

5. Deployment and environment configuration
Although deployment to a live server is not required for the academic submission, moving to any production-like environment later will require addressing configuration risks — such as securely managing secrets, handling CORS restrictions, and ensuring file upload paths are safe.

1.5 System Overview

The Bug Tracking and Reporting Tool is designed as a web-based system that enables teams to report, manage, and resolve software bugs in a structured and collaborative environment.



The workflow begins with user authentication, where new users can register an account and existing users can log in. Access to the system's features is determined by role-based permissions to ensure that actions such as project creation, team management, and bug assignment are controlled.

Once authenticated, users can create new projects and assign members to them. Projects act as a container for teams, which are created and managed within each project. Team creation allows administrators to group members based on skill sets or responsibilities, ensuring that bug resolution tasks can be effectively distributed.

Bugs can be reported within the context of a project and team. When a new bug is created, the system checks for duplicates using similarity detection algorithms. If similar or duplicate bugs are found, the system displays suggestions so that users can review issues appropriately. If no duplicate is detected, the process proceeds directly to assignment.

Assignment is managed in two stages. First, the system performs automatic team allocation based on the bug's labels. Second, it performs automatic user allocation by matching bug tags with stored user skills. This helps in assigning the most suitable person to resolve the issue.

Once a bug is assigned, the assignee can work on it, submit progress updates, add comments, and attach supporting files. Bugs can be moved through different status stages until they are either closed or reopened for further work. Planned enhancements to the system include a gamification feature, where users earn points for successful bug resolutions, and analytics and reporting capabilities to provide insights into team performance, bug trends, and overall project health.

The high-level flow of the system is illustrated in the above figure, which shows the progression from login to bug closure, including both implemented and planned features.

2. Background Research

2.1 Technical Research and Technology Justification

The Bug Tracking and Reporting Tool was developed using a modern full-stack architecture that combines a React-based frontend with a Spring Boot backend and PostgreSQL database. The choice of these technologies was driven by their maturity, active community support, and suitability for implementing the project's requirements, which include both core bug tracking capabilities and advanced features such as duplicate bug detection and automated team/user allocation.

2.1.1 Frontend: React, TypeScript, Tailwind CSS, shadcn/ui, and Radix

React was chosen for its component-based architecture, enabling modular and maintainable UI development. TypeScript adds static typing, improving reliability by catching type-related errors during development. Tailwind CSS was adopted for rapid, utility-first styling and consistent responsive design. The project uses shadcn/ui, a headless UI component library built on Radix UI primitives, to provide accessible, theme-consistent components such as modals, dropdowns, and tabs without sacrificing flexibility. This combination allowed for rapid UI assembly while maintaining full control over styling and behaviour.

Form management is handled by React-Hook-Form, with Zod for schema validation, ensuring that user input is validated before sending it to the backend.

2.1.2 Backend: Spring Boot (Java 17)

Spring Boot was selected for its robust ecosystem, quick setup, and seamless integration with Spring Security for authentication and role-based authorisation. The backend follows a Controller–Service–Repository pattern with Data Transfer Objects (DTOs) and Jakarta Bean Validation annotations for input validation. Java 17, being a Long-Term Support (LTS) release, provides access to the latest stable language features and performance optimisations.

2.1.3 Database: PostgreSQL with Flyway Migrations

PostgreSQL was chosen for its ACID compliance, scalability, and advanced indexing capabilities. Its support for JSON fields allows for flexibility in storing bug metadata alongside relational data. Flyway handles database migrations, ensuring that schema changes are version-controlled and consistently applied across environments.

2.1.4 Security: JWT Authentication and Role-Based Access Control

Authentication is implemented using JSON Web Tokens (JWT) for stateless, scalable session management. Spring Security enforces role-based access control (RBAC) to restrict operations such as project management, bug assignment, and configuration changes to authorised roles.

2.1.5 Advanced Feature Technologies

Duplicate Bug Detection

The duplicate detection mechanism is implemented entirely in Java, using a combination of text preprocessing, vectorisation, and similarity scoring:

- Apache Lucene's TF-IDF implementation is used to convert bug titles and descriptions into weighted term vectors.
- Cosine Similarity and Jaccard Index are implemented in-house to compare the TF-IDF vectors.
- Apache Commons Text provides the Levenshtein Distance algorithm for measuring edit distance between strings.
- Preprocessing steps such as case normalisation, tokenisation, and stop-word removal are handled using Apache Lucene's analysis tools.

This combination allows the system to detect duplicates with a balance of accuracy and performance, and the modular design means additional algorithms can be integrated in the future.

Automatic Team Assignment

The auto team allocation logic maps bug labels to specific teams within a project. When a new bug is reported, its labels are evaluated and matched to a predefined team mapping. This feature reduces manual assignment effort and ensures that bugs are directed to the most relevant team.

Automatic User Assignment

Automatic user allocation uses bug tags to match against user skill sets stored in the database. Skills are linked to users as structured attributes (e.g., Java, PostgreSQL). The allocation algorithm scores potential assignees based on tag/skill overlap and selects the highest-scoring match, with fallback options if there are ties or no perfect matches.

Technological Justification

The selected stack offers a balance of developer productivity, performance, and maintainability. React with shadcn/ui and Radix enables a professional, accessible UI without locking into rigid styling. Spring Boot and PostgreSQL provide a proven, enterprise-grade backend and database combination, while Flyway enforces database consistency. Advanced features like duplicate bug detection and automatic allocation are implemented directly within the backend using performant, well-maintained libraries such as Apache Lucene and Apache Commons Text, ensuring they integrate seamlessly with the rest of the system. This architecture is modular and extensible, making it suitable for future enhancements without significant rework.

2.2 Literature Review

To understand the landscape of bug tracking systems and to identify gaps that this project could address, several widely used tools were examined. The evaluation focused on their core features, strengths, limitations, and suitability for small to mid-sized development teams or open-source projects.

2.2.1 Jira

Jira, developed by Atlassian, is one of the most widely used issue and project tracking tools in the software industry. It offers extensive customisation options, agile board support, advanced reporting, and integration with other Atlassian products such as Confluence and Bitbucket. While Jira is feature-rich, its complexity can be overwhelming for smaller teams. The setup process can require significant configuration, and many advanced features are locked behind paid plans. For users who need a streamlined interface for core bug tracking tasks, Jira's depth can be both a strength and a barrier.

Strengths:

- Highly customisable workflows and fields.
- Rich reporting and dashboard capabilities.
- Strong ecosystem of plugins and integrations.

Limitations:

- Steeper learning curve for new users.
- Performance issues for very large projects without optimisation.

- Licensing costs can be prohibitive for small teams.

2.2.2 Bugzilla

Bugzilla is an open-source bug tracking system maintained by the Mozilla Foundation. It is known for its reliability and ability to handle large volumes of bugs. Bugzilla provides features such as advanced search queries, email notifications, and user group-based permissions. However, its user interface is dated compared to modern tools, and it lacks some of the advanced automation and analytics features found in newer platforms.

Strengths:

- Free and open-source with strong community support.
- Proven scalability for large projects.
- Powerful query capabilities.

Limitations:

- Outdated and less intuitive user interface.
- Limited built-in automation for tasks such as duplicate detection or auto assignment.
- Requires manual configuration for modern integrations.

2.2.3 Redmine

Redmine is a flexible, open-source project management and issue tracking tool. It supports multiple projects, role-based access control, Gantt charts, and a plugin ecosystem for extending functionality. Redmine's interface is cleaner than Bugzilla's but still lacks the polished, modern feel of newer SaaS solutions. Duplicate detection and automated assignment are not native features and require plugins or external scripts.

Strengths:

- Free and open-source with multi-project support.
- Good role management and permission control.
- Extensible via plugins.

Limitations:

- Limited built-in automation.
- UI can feel outdated.
- Plugin quality varies and may require maintenance.

2.2.4 GitHub Issues

GitHub Issues is integrated directly into the GitHub platform, making it convenient for teams already hosting code repositories there. It supports labels, milestones, project boards, and simple automation through GitHub Actions. However, its capabilities as a dedicated bug tracking tool are limited; it lacks native advanced features such as duplicate detection, complex reporting, or sophisticated assignment logic.

Strengths:

- Seamless integration with GitHub repositories and workflows.
- Easy to use with minimal setup.
- Supports automation via GitHub Actions.

Limitations:

- Limited reporting and analytics.
- No built-in duplicate detection or skill-based assignment.
- Less suitable for complex multi-team environments.

2.3 Identified Gaps and Justification for the Proposed System

From this evaluation, several common limitations emerge:

- Most tools require manual duplicate detection or rely on third-party plugins.
- Automatic team or user allocation based on bug metadata is generally absent or limited.
- Lightweight, accessible interfaces are often sacrificed for advanced functionality, or vice versa.

The proposed Bug Tracking and Reporting Tool aims to combine the usability of lightweight platforms with advanced features such as integrated duplicate bug detection using TF-IDF and similarity algorithms and automatic team/user allocation based on bug labels and skills. By focusing on a streamlined user experience, integrated automation, and a modular design, the system addresses the needs of teams that require efficiency without the overhead of overly complex configurations.

3. Requirements

3.1. Essential Requirements (Must have)

These requirements form the Minimum Viable Product (MVP) and must be delivered to ensure the system meets its core objectives.

1. **User Authentication and Authorisation:** Secure user registration and login using JSON Web Tokens (JWT) with role-based access control (Developer, Reporter, Admin).
2. **Backend API Implementation:** RESTful backend services developed using Spring Boot, incorporating validation and robust error handling.
3. **Database Management:** PostgreSQL database with schema versioning and migration handled by Flyway.
4. **Frontend Interface:** React, TypeScript, and Tailwind CSS-based interface with protected routes and secure token handling after authentication.
5. **Project Management:** Creation and management of projects, which serve as containers for teams and bug reports.
6. **Team Management:** Creation of teams within projects, with membership and role assignment, ensuring the team creator is assigned as Admin.
7. **Bug Reporting and Lifecycle Management:** Submission, updating, and tracking of bugs within projects, including the ability to change statuses.
8. **Logging:** Backend logging configuration to record and monitor key application events.

3.2. Required Requirements (Should have)

These requirements are important for improving system efficiency, accuracy, and user experience, but their absence would not prevent the system from functioning.

1. **Duplicate Bug Detection:** Identification of duplicate or similar bugs using a combination of TF-IDF vectorisation, Cosine Similarity, Jaccard Index, and Levenshtein Distance.
2. **Automatic Team Allocation:** Assignment of bugs to teams based on bug labels, with mappings configured in the system.
3. **Automatic User Allocation:** Assignment of bugs to individual users based on bug tags matched against stored user skills, with scoring and fallback logic.
4. **Project and Team Listing with Access Guards:** Enforcement of role and token-based access controls to protect restricted views and actions.

3.3. Optional Requirements (Could have)

These requirements enhance the functionality and usability of the system but are considered post-MVP features or stretch goals.

1. **Exporting Bug Data:** Ability to export bug lists and details in CSV or PDF format for reporting purposes.
2. **Bug Analytics and Dashboards:** Statistical and visual reporting on bug trends, resolution times, and team performance.
3. **Priority Levels and Classification:** Categorisation of bugs by priority (High, Medium, Low) and by type or category.
4. **Related Bug Suggestions and Smart Tagging:** Automatic suggestion of related bugs to assist triage.

5. **Activity Feeds and Audit Logs:** Project and team-level feeds showing significant actions and changes.
6. **Notification System:** Email or in-app notifications for assignments, status changes, or comments.
7. **Integration with GitHub Flows:** Linking bugs to specific commits, branches, or pull requests in external repositories.
8. **Gamification Features:** Points and leaderboards for successful bug resolution to encourage engagement.
9. **User Assignment Based on Workload:** Consideration of a user's current workload when allocating new bugs.

4. Technical Specifications

The Bug Tracking and Reporting Tool is a full-stack web application built with a React/TypeScript frontend and a Spring Boot backend backed by PostgreSQL. The system is designed for secure, modular, and extensible bug management with support for duplicate detection and automatic allocation.

4.1. Frontend

The frontend is implemented in React 18 with TypeScript, styled via Tailwind CSS. UI primitives come from shadcn/ui, which is built on Radix UI components (dialogs, dropdowns, tabs, switches), ensuring accessible, consistent behaviour while keeping styling flexible.

- **State & Forms**: react-hook-form with @hookform/resolvers and Zod for schema validation.
- **Routing**: react-router-dom with protected routes around authenticated areas.
- **HTTP**: axios with a central API client and JWT interceptor.
- **Utilities**: date-fns for date handling, clsx and tailwind-merge for class composition, class-variance-authority for variantable components, react-toastify for toasts, Framer Motion for subtle motion, react-day-picker for date inputs, Lucide icons.
- **Build/tooling**: Vite (with @vitejs/plugin-react), TypeScript, ESLint (typescript-eslint), @tailwindcss/vite.

This stack enables rapid UI assembly, strong type safety, and a clean component model that maps well to projects, teams, and bug flows.

4.2 Backend

The backend is written in Java 17 using Spring Boot 3.5 and follows a layered Controller --> Service --> Repository architecture with DTOs and Jakarta Bean Validation. Cross-cutting concerns include:

- **Security**: Spring Security with JWT (JJWT jjwt-api/jjwt-impl/jjwt-jackson) for stateless auth and role-based access control.
- **Data**: Spring Data JPA over PostgreSQL and Flyway for schema migrations.
- **Caching**: Spring Cache with Caffeine for hot lookups.

- **Querying & Utility**: QueryDSL JPA for type-safe ad-hoc queries, Slugify for human-readable slugs.
- **Observability**: Logback with structured output, Spring Boot Actuator enabled for health/metrics.
- **Dev**: Spring DevTools, Lombok (compileOnly/annotationProcessor) to reduce boilerplate.

Endpoints expose REST resources for authentication, users/roles, projects, teams, and bugs (lifecycle, comments, attachments), plus similarity checks and auto-assignment.

4.3 Database and Data Management

The system uses PostgreSQL database and objects are versioned with Flyway. Core entities include Users, Roles, Teams, Projects, Bugs, Labels/Tags, Skills, Comments, and Attachments. File attachments are saved on the server's filesystem with DB-stored metadata (path, owner, bug ID). Foreign keys enforce integrity.

4.4 Advanced Feature Implementations

4.4.1. Duplicate Bug Detection

Duplicate detection runs in-process (Java) and uses classic IR techniques:

- **Pre-processing**: Case normalisation, tokenisation, stop-word removal (simple pipeline in code).
- **Vectorisation**: TF-IDF built with Apache Commons Math for vector ops.
- **Similarity metrics**: Cosine Similarity and Jaccard Index (implemented in-house).
- **String distance**: Levenshtein via Apache Commons Text.
- **Persistence**: results and intermediate features can be cached/persisted to reduce repeat work.

This combination balances accuracy, speed, and simplicity without external microservices.

4.4.2. Automatic Team Allocation

On bug creation, label(s) are matched to a configurable label & team mapping within the project. The service remains simple to reason about and can be updated by admins without code changes (via config or admin UI).

4.4.3. Automatic User Allocation

Bug tags are matched to user skills stored in the DB. A scoring function ranks candidates by overlap, tie-breakers (e.g., fallback order) are planned to incorporate objective signals such as workload.

4.5. Security

Stateless JWT auth protects APIs, role checks guard sensitive actions (project/team admin, assignment). CORS is restricted by environment. DTO validation and consistent error handling reduce attack surface (XSS/SQLi mitigated via framework defaults and parameter binding). Secrets (DB creds, JWT key) are environment-specific and should be externalised for deployment.

4.6. Development Environment and Tooling

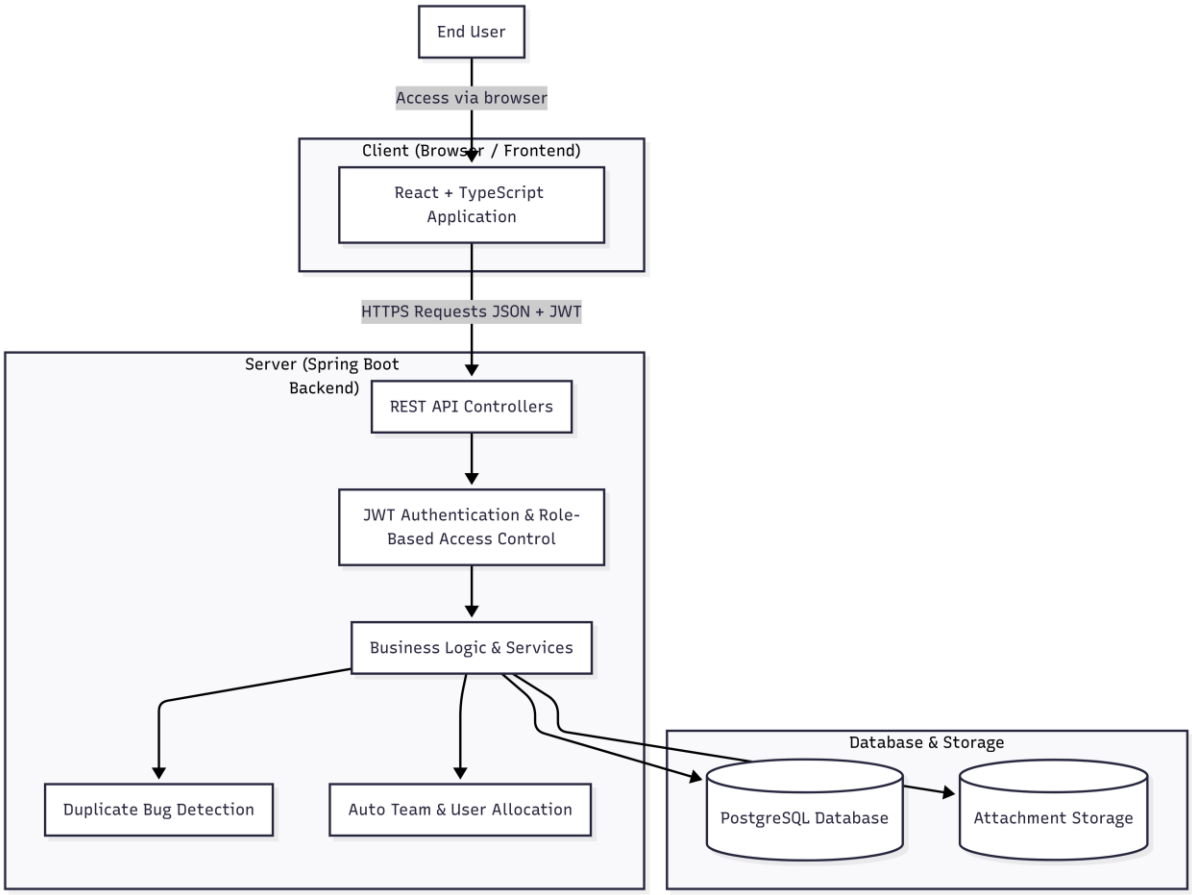
- **IDEs**: IntelliJ IDEA (backend), VS Code (frontend).
- **Version control & workflow**: GitLab with issues/branches/MRs, Postman & HTTPie for API checks.
- **Diagrams & docs**: Excalidraw, Mermaid live, Word for reports.

4.7. Deployment

Dev runs locally (Vite + Spring Boot JAR; PostgreSQL local). For delivery, the backend is packaged as a Spring Boot JAR; the frontend is built to static assets. Environment-specific profiles supply DB creds and JWT key. File storage path should be configured per environment.

5. Design

5.1. Client Server Architecture Design



The Bug Tracking and Reporting Tool follows a client–server architecture, where the application is split into three primary layers: the client (frontend), the server (backend), and the database & storage layer. This separation of concerns improves scalability, maintainability, and security, while allowing each layer to evolve independently.

Client (Browser / Frontend)
The client component is a React application written in TypeScript, running in the user's browser. It is responsible for rendering the user interface, handling user interactions, and communicating with the backend through secure HTTPS requests. The frontend uses JWTs (JSON Web Tokens) for authenticated API calls and provides role-based navigation to ensure that users only see and access functions appropriate to their permissions.

Server (Spring Boot Backend)
The server is implemented in Spring Boot and exposes a set of REST API controllers to the client. All requests pass through the security layer, which enforces JWT authentication and role-based access control (RBAC). Business logic is encapsulated within dedicated service classes that handle core operations such as project management, team creation, bug reporting, and lifecycle management.

Two advanced modules are integrated into the server layer:

- **Duplicate Bug Detection:** Uses TF-IDF vectorisation combined with similarity metrics (Cosine Similarity, Jaccard Index, Levenshtein Distance) to identify and suggest potential duplicate issues.
- **Auto Assignment:** Automates team and user allocation based on bug metadata (labels and tags) matched against configured mappings and stored user skill sets.

Database & Storage Layer
The backend persists application data in a PostgreSQL database, which stores structured entities such as users, projects, teams, bugs, labels, tags, and skills. Database schema versioning is handled via Flyway migrations to maintain consistency across environments. File attachments, such as screenshots or supporting documents, are stored in a dedicated file storage location, with metadata linking them to the relevant bug entries.

Data Flow

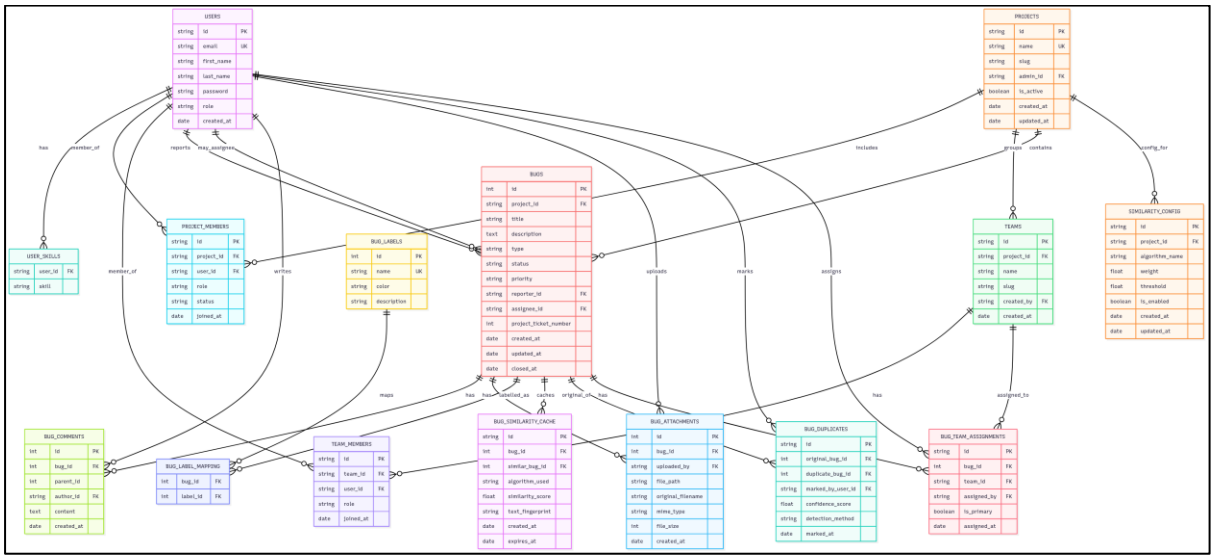
1. The end user interacts with the React frontend via a web browser.
2. The frontend sends HTTPS requests containing JSON payloads and JWTs to the backend API.
3. The backend authenticates the request, checks user permissions, and routes it to the relevant business service.
4. The service layer may query or update the database, store or retrieve attachments, and run duplicate detection or auto-assignment logic as required.
5. The backend returns structured JSON responses to the frontend, which updates the user interface accordingly.

This layered design ensures that the client is responsible solely for presentation and interaction, while the server handles all business rules, security enforcement, and data processing. The database and storage layer provide reliable, secure persistence for all application data and uploaded files.

5.2. Database Design

The database for the Bug Tracking and Reporting Tool is built using PostgreSQL and is organised to store all the important information about users, projects, teams, bugs, and related data. The design follows a relational model, meaning that information is stored in separate tables, and connections between them are managed through relationships.

At the heart of the system are the users, who can take part in projects and teams. Each project can have many members, and within a project, multiple teams can be created. Teams also have their own members, and this structure makes it easier to assign the right people to the right tasks.



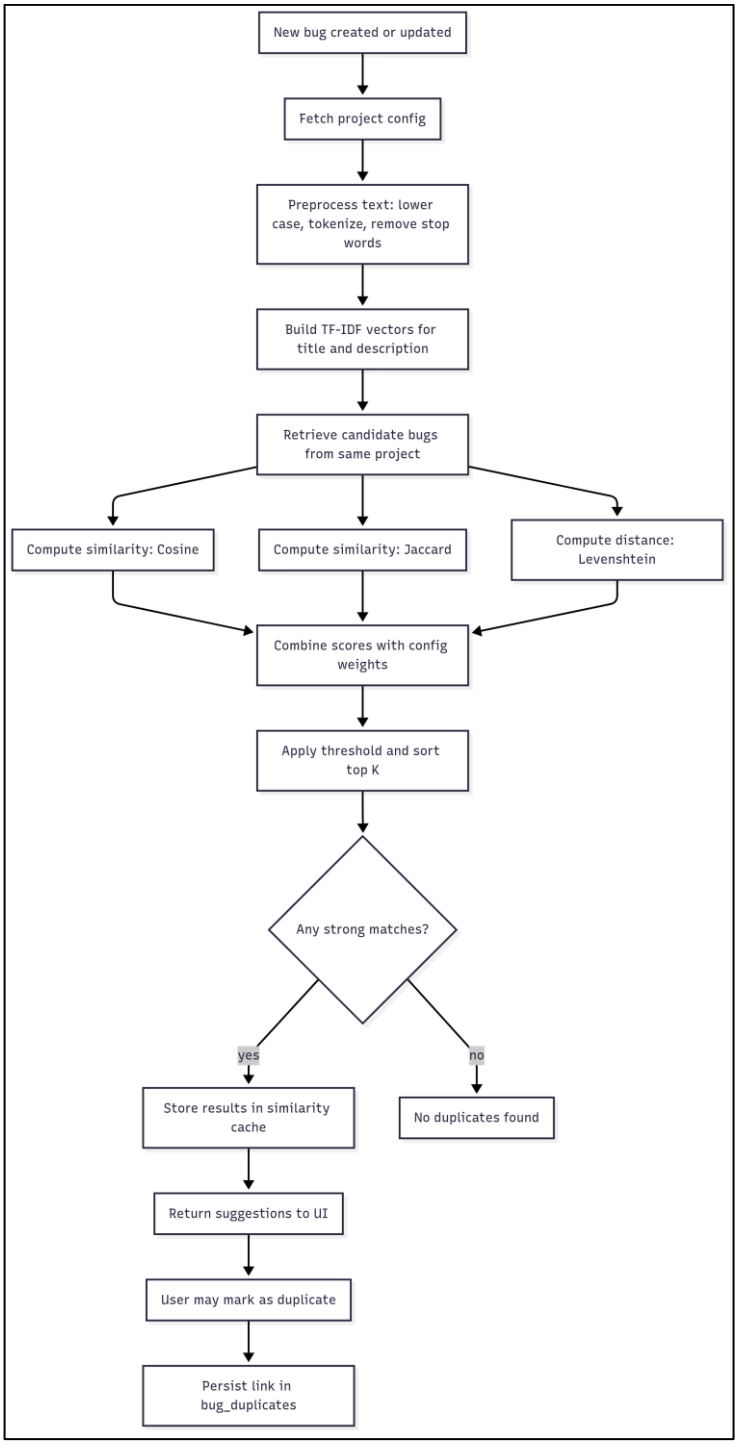
Bugs are linked to projects and can have various details such as a title, description, type, status, and priority. Each bug can have comments, file attachments, and labels for easier categorisation. The system also supports linking bugs to one or more teams and assigning them to specific users.

To make bug management more efficient, there are special tables for duplicate bug detection, which store records of similar bugs found by the system, and for automatic team allocation, which records how bugs were assigned based on their labels or tags.

The database also has configuration tables, such as those storing the similarity detection settings for each project. All files, such as screenshots or documents related to a bug, are stored on the server, with only the file details (like path and name) kept in the database.

This design ensures that information is well organised, easy to search, and flexible enough to support both the core bug tracking features and the more advanced tools like duplicate detection and automatic assignment.

5.3. Duplicate Bug Detection Design



1. **When a new bug is created or an existing one is edited:** As soon as the user submits a bug (title and description), the backend calls the duplicate-check service. This happens before any assignment logic, so the user can see suggestions early.
2. **Read the project settings:** Each project can choose which similarity methods to use and what score counts as a "good match". The service loads that configuration so it can apply the right weights and threshold for this project.

3. **Prepare the text:** The system cleans the input text so comparisons are fair. It lowercases the text, splits it into words, and removes common stop words. This reduces noise and makes results more stable.
4. **Build TF-IDF vectors:** The cleaned title and description are turned into TF-IDF vectors. This gives more weight to words that are important in this project and less weight to very common words.
5. **Pick candidate bugs to compare:** Only bugs from the same project are considered. The service fetches a shortlist of recent or relevant bugs instead of scanning everything, which keeps the check fast.
6. **Calculate similarity in three ways:** For each candidate, the service computes several scores:
    a. cosine similarity on the TF-IDF vectors (measures how close two texts are as "angles"),
    b. jaccard similarity on token sets (measures word overlap),
    c. levenshtein distance on strings (measures how many edits would turn one string into another).

    These measures catch different kinds of matches: near-identical wording, shared key terms, or small spelling changes.

7. **Combine and rank results:** The service combines the individual scores using the weights from the project settings, then sorts the list and keeps the top results. It also applies the project threshold so only strong matches remain.
8. **Cache suggestions:** Strong matches are written to a small cache table. If the same bug is edited again soon, the system can reuse the results without recomputing everything.
9. **Return suggestions to the UI:** The API returns the top suggestions to the frontend. The bug form shows possible duplicates with their scores so the reporter can decide what to do next.
10. **Mark as duplicate (optional):** If the user confirms that two bugs are the same, the system saves a link in the duplicates table. This keeps the record of which bug is the original and which one is the duplicate.

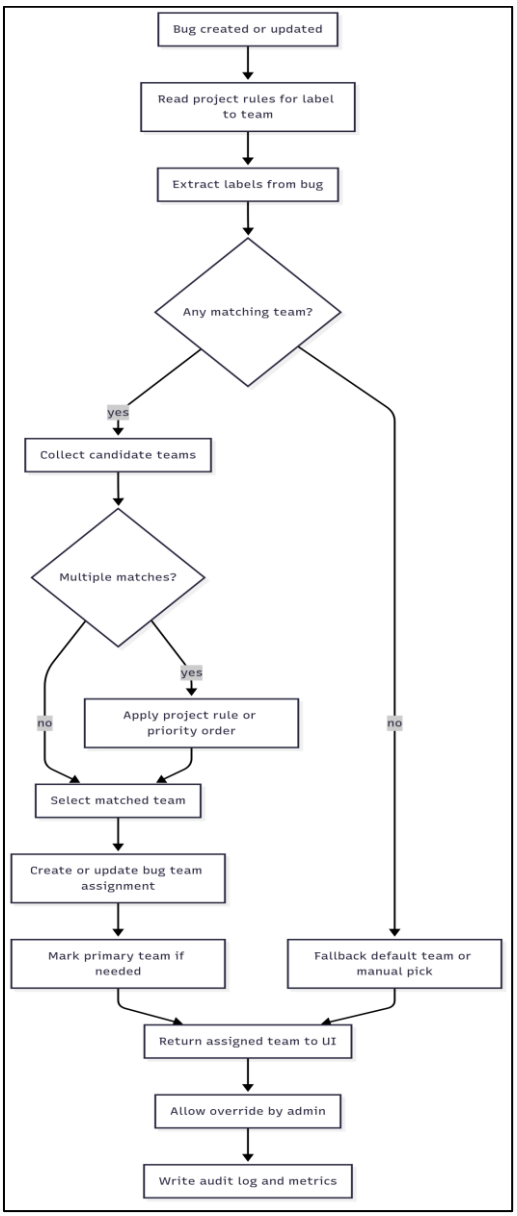
Why this design works?

1. It runs inside the main backend, so there is no extra service to deploy.
2. TF-IDF plus multiple similarity checks gives better coverage than any single method.
3. Per-project settings let teams adjust the sensitivity if they report many similar bugs.
4. Caching avoids repeat work and keeps the form responsive.

Current limitations and next steps

1. The quality of suggestions depends on clean text and good thresholds; we plan to add more sample data for tuning.
2. The candidate shortlist is basic; adding smarter prefilters (by label, component, or recent activity) can speed it up further.
3. We plan to add simple tests with real bug pairs to guard against regressions when we adjust the thresholds or weights.

5.4. Auto Team Assignment To Bug



When a new bug is reported in the system, one of the first steps after saving it is to decide which team should work on it. The auto team assignment feature was designed to make this process faster and more consistent, especially in projects with many teams.

The process starts by checking the labels attached to the bug. Each project can define rules that link certain labels to specific teams. For example, a bug with the label “Frontend” could be automatically assigned to the “Frontend Team”, while “Database” might go to the “Backend DB Team”.

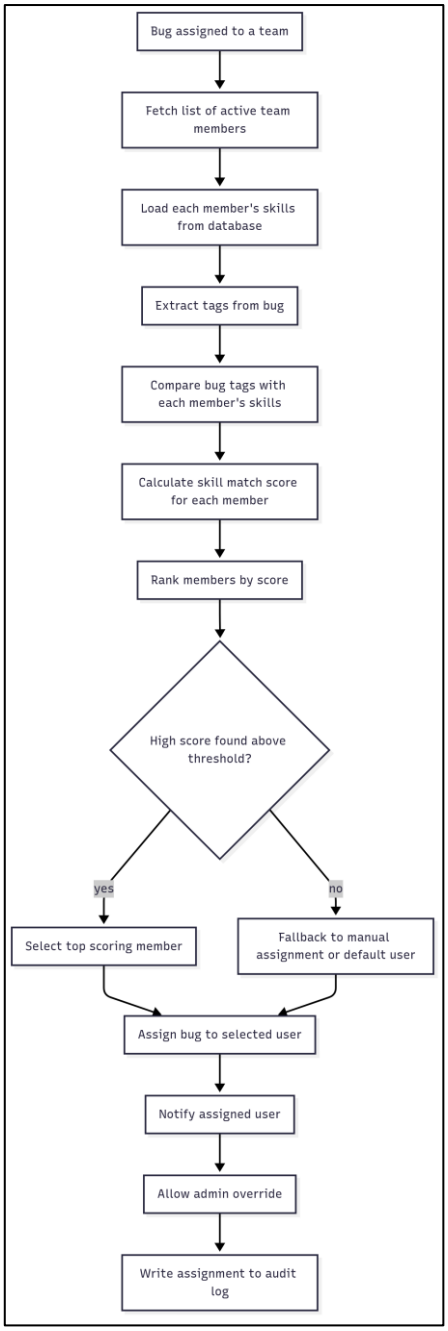
If one or more teams match the bug’s labels, the system then decides which team to choose. If there’s only one match, that team is assigned directly. If there are multiple matches, the system follows the project’s configured rules or a priority order to select the most appropriate team.

Once the team is chosen, the system creates or updates the record linking the bug to that team. In some cases, a bug can be linked to multiple teams, but one will be marked as the “primary” team responsible for it. This information is then returned to the frontend so the reporter or admin can see it right away.

If no labels match any team rules, the system can either assign the bug to a default fallback team or leave it for manual assignment by a project admin. In all cases, any assignment made by the system can be overridden by an admin, and an audit record is kept so changes can be tracked later.

This design ensures that bugs are routed quickly to the right group of people without relying entirely on manual input, while still keeping flexibility for exceptions and special cases.

5.5. Auto user assignment based on skills



The goal is to pick the most suitable person from the right team without making the reporter choose manually every time. This saves time and keeps assignments consistent.

When a bug is routed to a team, the system first loads the active members of that team. For each member, it pulls their saved skills from the database. Examples are Java, React, PostgreSQL, or other tags you have recorded for that user.

From the bug, the system reads the tags that describe the work needed. These tags usually come from the form where the reporter selects technologies or areas related to the issue.

The service then compares the bug's tags with each user's skills. Every exact match increases that person's score. Partial matches can add a smaller amount if configured. The idea is simple: the more overlap, the higher the score.

After scoring, the members are ranked. If the best score is above a basic threshold, the top person is selected automatically. If no one reaches the threshold, the system falls back to manual assignment or to a safe default user set by the project.

The assignment is written back to the bug record, and the UI is updated so the reporter and team can see who was chosen. Admins can always change the assignee. When an override happens, the system records it so there is a clear history of who changed what and when.

Edge cases are handled in a straightforward way. If two people have the same score, the current version uses a simple tie break (for example, a stable sort order). If a team member has no skills on record, they will not score highly until their profile is updated. If the team has no active members, the feature exits early and asks for manual assignment.

This first version focuses on matching by skills and bug tags. It works best when user skills are kept up to date and bugs are tagged correctly. The plan is to add extra signals later, such as current workload or recent activity, so ties are resolved more fairly and busy people are not overloaded.

Right now this feature is partially implemented and needs more testing with real data. The next steps are to add a few test cases, tune the threshold, and finalise the tie-break rule. After that, we can add a simple admin screen so project leads can update skills and tag lists without touching the database.

6. Implementation

6.1. Duplicate Bug Detection

This feature helps the reporter see possible duplicates while creating or editing a bug. It runs inside the backend and returns suggestions to the UI in real time.

Where the check is triggered

- The frontend calls a REST API after the user types the title and description, and again when the user submits the form.
- The backend receives the request, runs the duplicate check, and returns the top matches with their scores.

```java
BugController.java ×
backend > src > main > java > com > pbm5 > bugtracker > controller > BugController.java > BugController
42  public class BugController {
159         /**
160          * Create a new bug with similarity pre-check
161          * This endpoint includes automatic duplicate detection before creation
162          */
163         @PostMapping("/with-similarity-check")
164         public ResponseEntity<?> createBugWithSimilarityCheck(
165                 @PathVariable String slug,
166                 @Valid @RequestBody CreateBugRequest request,
167                 @RequestParam(required = false, defaultValue = "false") boolean proceedAnyway,
168                 Authentication authentication) {
169
170             logger.debug(format:"createBugWithSimilarityCheck method called for project: {}", slug);
171
172             UUID userId = getCurrentUserId(authentication);
173             UUID projectUuid = getProjectUuidFromSlug(slug, userId);
174
175             // Validate user has access to create bugs in this project
176             bugSecurityService.validateBugCreateAccess(projectUuid, userId);
177
178             // If not proceeding anyway, check for similar bugs first
179             if (!proceedAnyway) {
180                 try {
181                     // This would require BugSimilarityService to be injected
182                     // For now, we'll create the bug directly
183                     logger.debug(msg:"Similarity check would be performed here");
184
185                     // TODO: Implement similarity check
186                     // List<BugSimilarityResult> similarBugs = bugSimilarityService.findSimilarBugs(
187                     // request.getTitle(), request.getDescription(), projectUuid);
188                     //
189                     // if (!similarBugs.isEmpty()) {
190                     // Map<String, Object> response = Map.of(
191                     // "hasSimilarBugs", true,
192                     // "similarBugs", similarBugs,
193                     // "message", "Similar bugs found. Review them or proceed anyway."
194                     // );
195                     // return ResponseEntity.status(HttpStatus.CONFLICT).body(response);
196                     // }
197                 } catch (Exception e) {
198                     logger.warn(format:"Similarity check failed, proceeding with bug creation: {}",
199                             e.getMessage());
200                 }
201             }
```

```java
        // Create the bug if no similar bugs found or user chose to proceed anyway
        Bug bug = bugService.createBug(
                projectUuid,
                request.getTitle(),
                request.getDescription(),
                request.getType(),
                request.getPriority(),
                userId, // reporter
                request.getAssigneeId(),
                request.getLabelIds());

        logger.debug(format:"Bug created successfully with ID: {}", bug.getId());
        return ResponseEntity.status(HttpStatus.CREATED).body(new BugResponse(bug));
}
```

Project-level configuration

- Each project can choose which algorithms are active and the threshold for a “good match”.
- The service reads the configuration before scoring.

```java
@Repository
public interface SimilarityConfigRepository extends JpaRepository<SimilarityConfig, UUID> {

    // === Basic Configuration Queries ===

    /**
     * Find all configurations for a specific project
     */
    @Query("SELECT sc FROM SimilarityConfig sc WHERE sc.project.id = :projectId " +
            "ORDER BY sc.algorithmName")
    List<SimilarityConfig> findByProjectId(@Param("projectId") UUID projectId);
```

Preprocessing

- The text is normalised: lowercasing, tokenising, removing stop words, and trimming extra spaces.
- This step reduces noise so different titles with similar meaning can still match.

```java
/**
 * Preprocess text for similarity analysis by cleaning and normalizing
 *
 * @param text Raw text input
 * @return Preprocessed text ready for similarity calculation
 */
public String preprocessText(String text) {
    if (text == null || text.trim().isEmpty()) {
        return "";
    }

    String processed = text.toLowerCase().trim();

    // Remove code snippets (they often contain non-meaningful content for
    // similarity)
    processed = CODE_SNIPPET_PATTERN.matcher(processed).replaceAll(replacement:" ");

    // Remove URLs and email addresses
    processed = URL_PATTERN.matcher(processed).replaceAll(replacement:" ");
    processed = EMAIL_PATTERN.matcher(processed).replaceAll(replacement:" ");

    // Remove special characters but preserve spaces
    processed = SPECIAL_CHARS_PATTERN.matcher(processed).replaceAll(replacement:" ");

    // Normalize whitespace
    processed = MULTIPLE_SPACES_PATTERN.matcher(processed).replaceAll(replacement:" ");

    return processed.trim();
}
```

TF-IDF vector building

- The service converts the cleaned text into TF-IDF vectors.
- Apache Commons Math is used for vector operations.
- Term frequencies for the new bug are combined with IDF values learned from the project's existing bugs.

Candidate selection

- To keep things fast, only bugs from the same project are considered.
- A shortlist is fetched (for example, recent bugs or those with overlapping words).

Similarity scoring

- For each candidate bug:
  - cosine similarity is calculated on the TF-IDF vectors,
  - jaccard similarity is calculated on token sets,
  - levenshtein distance is calculated on strings.
- Apache Commons Text provides the Levenshtein distance; cosine and jaccard are implemented in code.

```java
/**
 * Calculate cosine similarity using Apache Commons Text.
 *
 * Cosine similarity measures the angle between two vectors in a
 * multi-dimensional space.
 * It's particularly effective for comparing documents of different lengths as
 * it
 * normalizes for document length.
 *
 * Citation: Apache Software Foundation. (2024). "CosineSimilarity Class
 * Documentation."
 * Apache Commons Text API.
 * https://commons.apache.org/proper/commons-text/apidocs/org/apache/commons/text/similarity/CosineSimilarity.html
 *
 * @param text1 First text to compare
 * @param text2 Second text to compare
 * @return Cosine similarity score (0.0 to 1.0)
 */
public double calculateCosineSimilarity(String text1, String text2) {
    if (text1 == null || text2 == null || text1.trim().isEmpty() || text2.trim().isEmpty()) {
        return 0.0;
    }

    // Preprocess texts
    String processed1 = textPreprocessor.preprocessText(text1);
    String processed2 = textPreprocessor.preprocessText(text2);

    if (processed1.isEmpty() || processed2.isEmpty()) {
        return 0.0;
    }

    // Create frequency maps for Apache Commons Text CosineSimilarity
    Map<CharSequence, Integer> vector1 = textPreprocessor.createCharacterFrequencyMap(processed1);
    Map<CharSequence, Integer> vector2 = textPreprocessor.createCharacterFrequencyMap(processed2);

    if (vector1.isEmpty() || vector2.isEmpty()) {
        return 0.0;
    }

    // Use Apache Commons Text CosineSimilarity
    return cosineSimilarity.cosineSimilarity(vector1, vector2);
}
```

Combining scores and applying threshold

- Scores are combined using the project's configured weights.
- Results are sorted, and a threshold is applied so only strong matches remain.

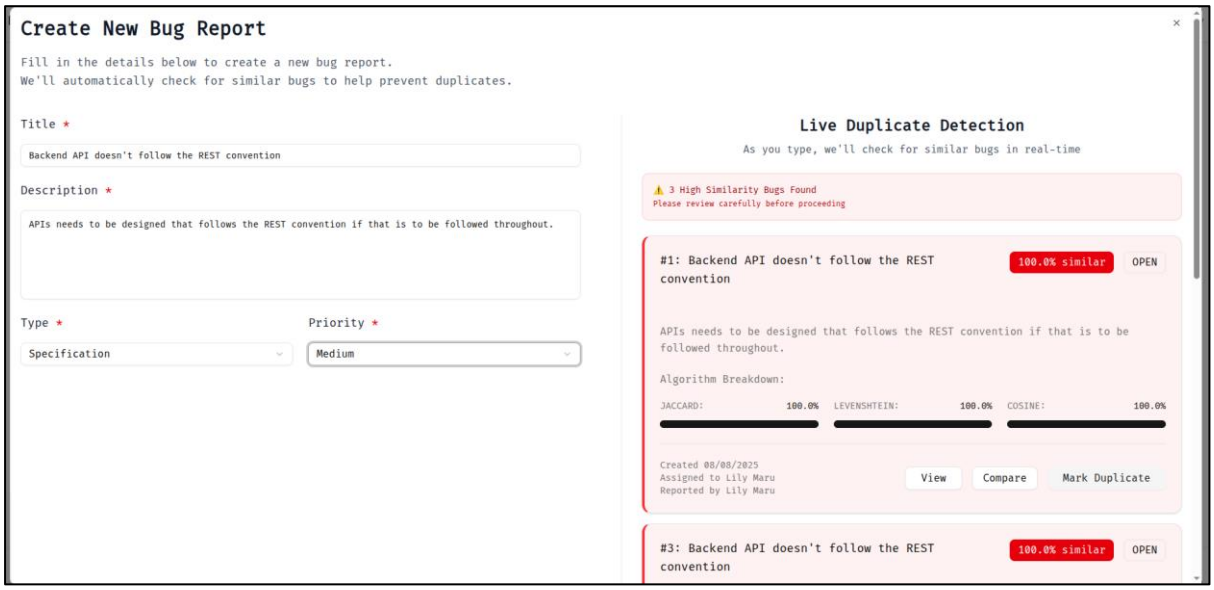
Caching and returning results

- Top results are saved to a small cache table so the next request for the same draft is faster.
- The API returns a list with bug id, title, status, and the three individual scores plus the combined score.

Frontend display

- The right-hand panel shows live suggestions while the user types.
- For each suggestion, the UI shows the combined score and the breakdown for jaccard, levenshtein, and cosine.

- Actions include view, compare, and mark duplicate.



Why this implementation was chosen

- Everything runs in the main backend, so there is no extra microservice to manage.
- TF-IDF plus multiple similarity measures gives better results than any one method.
- Per-project weights and threshold let teams tune sensitivity.
- Caching reduces latency during typing.

Current limitations and next steps

- The shortlist filter is simple. Adding pre-filters by label or component could make it even faster.
- Thresholds need tuning with real data. We will create a small test set of known duplicates.
- We plan to add unit tests for each metric and an integration test for the full flow.

7. Critical Reflection and Evaluation

7.1. Challenges faced during development

During the development of the Bug Tracking and Reporting Tool, several challenges were encountered that shaped the design and implementation process. These challenges highlighted the importance of correct module sequencing, managing dependencies, and aligning the frontend and backend from the start.

One of the most significant challenges was implementing the Teams module before the Projects module. In the intended design, teams are a sub-module within projects and are tightly coupled to them through both logic and database relationships. Building the Teams module first meant there was no established project structure to attach teams to. This led to extensive refactoring

once the Projects module was introduced. The refactoring involved rewriting parts of the API, updating service logic, modifying frontend calls, and making substantial database schema changes to correctly establish foreign keys and relationships. This experience underlined the importance of planning module dependencies early to avoid rework.

Another challenge was balancing the scope of work with the project timeline. The plan included both core features and advanced functionalities, but time constraints required prioritising essential modules over optional ones. This meant that some advanced features such as gamification and analytics were postponed to later stages, while partial implementations like auto user assignment were earmarked for refinement.

Synchronisation between the frontend and backend was also a recurring difficulty. Any change to an API endpoint, data contract, or validation rule required corresponding updates on the frontend. This often led to periods of blocked work while waiting for changes to be tested and merged on both sides.

The learning curve for certain technologies added complexity. On the frontend, mastering Tailwind CSS in combination with ShadCN UI components required additional time for research and experimentation. On the backend, implementing duplicate bug detection involved integrating TF-IDF vectorisation with Apache libraries and tuning similarity algorithms to produce accurate results without excessive processing time.

Database evolution was another area of challenge. Schema changes during refactoring, especially after re-ordering module implementations, required multiple migration scripts and data adjustments. Coordinating these changes while preserving existing test data demanded careful planning and version control discipline.

Finally, testing practices had to be adapted to fit the available time. While manual testing confirmed that core flows worked end-to-end, automated testing particularly for frontend interactions remained limited. Setting up a more comprehensive automated testing framework is planned for the later stages to reduce regression risk.

Despite these challenges, each obstacle led to a clearer understanding of the system architecture and dependencies, resulting in a more maintainable and extensible design going forward.

8. Conclusion

The development of the Bug Tracking and Reporting Tool has resulted in a functional and extensible system that addresses the core needs of managing software bugs in a structured and efficient way. The project successfully implements core modules such as user authentication, project creation, team management, bug reporting, and basic bug lifecycle management. In addition, advanced features such as duplicate bug detection and auto team assignment have been implemented, while auto user assignment is partially complete and planned for refinement.

The project has been built with a layered architecture, using React and TypeScript for the frontend, Spring Boot for the backend, and PostgreSQL for persistent storage. This

combination has provided a balance between maintainability, scalability, and developer productivity. Integration between these components has been achieved through a set of REST APIs secured with JWT authentication and role-based access control.

Although certain advanced features such as gamification, analytics, and complete automated testing remain pending, the current system lays a strong foundation for these enhancements. Challenges such as module dependency mis ordering, database schema refactoring, and synchronisation between frontend and backend were addressed successfully, resulting in a more robust design.

Overall, the tool demonstrates that with the right architectural choices, even a complex system like a bug tracker can be developed incrementally while keeping it adaptable to new features. The work completed so far ensures that future additions can be integrated without major redesigns.

9. Recommendations for future work

The following recommendations aim to guide the next stages of development and help improve the functionality, performance, and maintainability of the system:

1. **Complete pending advanced features:** Implement the gamification module, analytics dashboard, and fully functional auto user assignment. These will enhance user engagement, provide actionable insights, and further automate task allocation.
2. **Refine duplicate bug detection:** Optimise preprocessing and similarity algorithms for better performance on larger datasets. Introduce pre-filters based on labels or components to reduce candidate search space.
3. **Enhance auto user assignment:** Incorporate workload balancing and recent activity tracking to avoid overloading individual team members. Allow project admins to adjust matching rules via a UI rather than database changes.
4. **Implement robust automated testing:** Develop unit, integration, and end-to-end tests for both frontend and backend to improve regression safety. Include performance tests for advanced features such as duplicate detection.
5. **Improve database migration strategy:** Strengthen schema versioning and change management to minimise disruption during feature updates and refactoring.
6. **Enhance UI/UX:** Expand use of pre-built UI component libraries, improve responsiveness for mobile devices, and streamline complex workflows such as bulk bug updates or mass assignment.
7. **Add notification systems:** Implement in-app and email notifications for key events such as bug assignment, status changes, and duplicate confirmations.
8. **Performance and scalability improvements:** Introduce caching for frequently accessed project data, optimise queries, and explore asynchronous processing for heavy tasks like similarity scoring.

By following these recommendations, the system can evolve into a feature-rich, high-performance platform capable of supporting larger teams and more complex projects without sacrificing usability or maintainability.

10. References

[1] Cabrel, E. C. Tiogo, "Implement JWT authentication in a Spring Boot 3 application," *Medium*, Mar 19, 2024. Available at: https://medium.com/@tericcabrel/implement-jwt-authentication-in-a-spring-boot-3-application-5839e4fd8fac

[2] Baeldung, "Spring Security with JWT," *Baeldung*, accessed Aug 2025. Available at: https://www.baeldung.com/spring-security-jwt

[3] Baeldung, "Guide to Spring Security Role-Based Access Control," *Baeldung*, accessed Aug 2025.
Available at: https://www.baeldung.com/role-and-privilege-for-spring-security-registration

[4] Baeldung, "The SOLID Principles in Java," *Baeldung*, accessed Aug 2025. Available at: https://www.baeldung.com/solid-principles

[5] DevSkillBuilder, "Understanding Spring Security," *Dev Skill Builder*, Jan 29, 2024. Available at: https://www.devskillbuilder.com/understanding-spring-security-b57571bfb988

[6] Tunwar, P., "TF-IDF in NLP (Term Frequency–Inverse Document Frequency)," *Medium*, 2023.
Available at: https://medium.com/@abhishekjainindore24/tf-idf-in-nlp-term-frequency-inverse-document-frequency-e05b65932f1d

[7] Programming Historian, "Analyzing Documents with TF-IDF," *Programming Historian*, May 13, 2019. Available at: https://programminghistorian.org/en/lessons/analyzing-documents-with-tfidf

[8] Giovannini, P., "Best Practices for Query Optimization in PostgreSQL," *TigerData Blog*, Nov 28, 2024. Available at: https://www.tigerdata.com/blog/best-practices-for-query-optimization-in-postgresql

[9] EDB Team, "PostgreSQL Query Optimization with EXPLAIN ANALYZE," *EnterpriseDB Blog*, Sep 18, 2024. Available at: https://www.enterprisedb.com/blog/postgresql-query-optimization-performance-tuning-with-explain-analyze

[10] Pal, P., "Mastering PostgreSQL: From Beginner to Advanced Query Optimization," *Medium*, Mar 31, 2025. Available at: https://medium.com/@ppal4536/mastering-postgresql-from-beginner-to-advanced-query-optimization-ec3a7305d39b

[11] Ferretti, M., "Optimizing PostgreSQL queries: 12 indexing pitfalls and how we fixed them," *Medium*, May 21, 2025. Available at: https://medium.com/cubbit/optimizing-postgresql-queries-12-indexing-pitfalls-and-how-we-fixed-them-81c25615a84e

[12] Kumarasinghe, M., "Secure Your Spring Boot Application with Spring Security," *Code.likeagirl*, Nov 28, 2024. Available at: https://code.likeagirl.io/secure-your-spring-boot-application-with-spring-security-9ea4d3aa71b7